

APUNTES DE RUST

DECLARACIÓN DE VARIABLES

La declaración de la variable es inversa a c:

Nombre_de_variable: tipo

var1: f32

var2: f64

Los tipos en comparativa:

int	-> i32	char	-> char (son 4bytes unicode no solo ascii)
-----	--------	------	--

long	-> i64	bool	-> bool
------	--------	------	---------

float	-> f32	char*	-> &str y &'static str (*nota)
-------	--------	-------	--------------------------------

double	-> f64	std::string	-> String
--------	--------	-------------	-----------

U-Int -> u32

U-long -> u64

NOTA: en C se puede devolver un `char*` pero si se hace un `char str[] = "hola"` y se devuelve eso, compila pero tenemos un puntero fantasma, por que dicho puntero muere en la función. Rust está obsesionado con la seguridad y esto no lo permite. Por eso, cuando nuestra función devuelva un "char" tendremos que usar el `&'static str`, y cuando lo lea de un parametro donde morirá dentro de la función podremos usar un `&str`.

El ' del &' indica lifetime y se puede ver más abajo qué significa.

Para asignar un valor a una variable seria:

```
let a = 32; //no habría una declaracion previa tipo a: i32, y Rust deduce lo que es.\
```

```
let a = 32 as f32 //le indicamos que es un float
```

o

```
let a: i32 = 42;
```

LET es para hacer que no pueda variar (NO PUEDE SER GLOBAL). **LET MUT** para que varíe (Hay que poner **let** delante), y **CONST** para constante (si puede ser global)

```
let mut i = 0; //tipico para hacer while ya que la i variará. No global. En runtime
```

```
const i: i32 = 0; //no puede mutar NUNCA en compilacion y puede ser global. Necesita ser declarado el tipo. y se crea en tiempo de compilación
```

Si a una variable le ponemos '_' delante, esta variable aunque no se use, el compilador no se quejará de ella:

```
let _num: f32 = 0.4; //aunque no la use el compilador no lanzara un warning
```

VARIABLES DUPLAS

Rust puede almacenar varias variables en duplas que pueden tener dos o más tipos de datos. Por ejemplo:

```
let dupla1: (i32, i32) = (32, 42);  
let dupla2: (i32, bool, &str) = (32, true, "hola");
```

De tal forma que si queremos acceder a sus elementos seria:

```
if dupla2.2 == "hola" {...}
```

VARIABLES GLOBALES

El tener una variable global en Rust es un problema, ya que puede haber dataraces para modificarla así que tiene que estar protegida.. si no no deja.

La forma clásica es:

```
static mut A: i32 = 0; //variable global  
  
fn main(){  
    if true{  
        unsafe{ A = 10;} //no se recomienda pero permitido. unsafe le dice el compilador que confie en el programador.  
    }  
}
```

Unsafe es un peligro.. así que usamos **atomic** para no provocar dataraces:

```
use std::sync::atomic::{AtomicI32, ordering}  
  
static A: AtomicI32 = AtomicI32::new(0); //no hace reserva de memoria. Solo inicializa a cero  
  
fn main() {  
    if true {  
        A.store(10, Ordering::Relaxed); //Relaxed es el nivel más débil del ordenamiento atómico.  
solo  
me importa modificar el valor, no el orden del tiempo en que se haga.  
    }  
}
```

PUNTEROS A FUNCION

Si tenemos una función:

```
fn suma(a: i32, b: i32) -> i32 { a + b };
```

el puntero a funcion sería:

```
let f = fn(a, b) -> i32 = suma;
```

y se usaría como:

```
let resultado = f(4, 2);
```

Si lo recibimos como argumento de la función:

```
fn procesar_funcion<T>(f: fn(&T) -> bool (o lo que sea));
```

PANIC!()

Rust cuando encuentra un proceso que no puede solventar (por ejemplo dividir por cero) entra en **Panic** liberando toda la memoria y saliendo limpiamente. Para forzarlo se usa esto:

```
fn set_title(&mut self, title: String){ //esto es un setter de una estructura por eso lleva el self
    if title.is_empty(){panic!("El título no puede ser vacío");} //panic hará salirse el programa
    if title.len() > 50 {panic!("El título es más largo de 50 bits");}
    self.title = title
}
```

ExitCode

Como en C, podemos retornar un valor de retorno 1 o 0 del programa. Para ello usamos el `ExitCode` de esta manera:

```
use std::process::ExitCode;

fn main() -> ExitCode{
    ...
    ExitCode::from(0)
}
```

Result()

Rust la forma de devolver valores es muchas veces un `Result()`, que es una pareja donde el primero es un `Ok(valor)`, y el segundo un `Err(error)`. Es como un interruptor de dos posiciones

```
// Devuelve un Ok con el resultado (f32) O un Err con un texto (&'static str)

fn dividir(dividendo: f32, divisor: f32) -> Result<f32, &'static str> {
    if divisor == 0.0 {
```

```

        return Err("¡No se puede dividir entre cero!"); // Pasó algo malo
    }
    Ok(dividendo / divisor) // Todo salió bien }

fn main() {
    // Cómo usar el resultado con un 'match' limpio
    match dividir(10.0, 2.0) {
        Ok(resultado) => println!("El resultado es: {}", resultado),
        Err(mensaje) => println!("Error: {}", mensaje),
    }
}

```

Vamos a verlo con una forma más compleja. Abriendo un archivo que puede dar errores:

```

fn process_file(str: &str) -> Result<(), &'static str>{ //() no nos interesa el tipo devuelto. Un void.
    if let Ok(input_file) = File::open(str){ //tras el open es un Result<File, std::io::error>
        let _reader = BufReader::new(input_file);
        for line in _reader.lines(){
            let line = match line{
                Ok(value) => value,
                Err(e) => return Err("Error: Error leyendo el archivo\n"),
            };
        }
    }
    return Ok(());
} else {
    return Err("Error: No puede abrir el fichero\n");
}
}

```

En este caso `_reader.lines()` devuelve un ITERADOR `line` y cada uno de ellos devuelve un `Result<String, error>` para averiguarlo hemos usado el `match`.. pero se puede hacer más pa línea:

```
let line = line?;
```

Esto lo que hace es preguntar con el `?`, si es `Ok`, suelta el `line` y si no un `Err`, pero dicho `Err` sería del tipo `std::io::error` y no `&'static str` como lo que devuelve por lo que el compilador fallaría.

Para solucionarlo, debemos mapear el error con `map_err`, pero este método necesita una función que recibe el error `std::io::error` y devuelve un `&str`. Le decimos que nos da igual lo que recibimos, y solo devuelva el string. Para ello usamos una función closure, detallana más abajo en los apuntes y ponemos la línea como:

```
let line = line.map_err(|_|, "Error: Error leyendo el archivo\n");
```

ESTRUCTURAS

Rust no tiene objetos ni clases, pero sí que puede acercarse a ellos con las estructuras. En Rust todo es privado que se declare. Para hacerlo accesible hay que ponerle la palabra **pub** delante. No solo a la estructura.

(fichero.rs)

```
pub struct Prueba{ //debe ser CamelCase P mayúscula
    pub var1: i32,
    pub var2: char, //si no son pub estas variables no se puede acceder a ellas
}
```

Para declararlas hay que hacer así, ya que tiene que declararse en la línea:

```
let kk = Prueba { n1: 32, str: 'b', }
```

Si la estructura está en otro archivo (modulo) lo podemos Importar el **fichero.rs** como

(fichero: main.rs)

```
mod fichero; //importa el fichero fichero.rs
fn main(){
    let a = fichero::prueba { var1: 42, var2: 'a'};
}
```

Ahora imaginemos que tenemos un archivo.rs que tiene esto:

(fichero.rs)

```
pub struct Prueba{ //para implementar el getter tiene que ser la estructura pub
    x: i32,
}
pub fn test() -> Prueba{ //-> devuelve un tipo Prueba (estructura)
    Prueba {x: 56}
}
```

No habría problema en darle valor dentro de fichero.rs PERO si queremos acceder a él, tendríamos que hacer un método de la estructura en el propio archivo fichero.rs:

```
impl Prueba{
    pub fn get_x(&self) -> i32{ //tiene que ser pub también. -> i32 = devuelve un int.
```

```

//&self es como los objetos.. o this en c++ va implicito
(self) lo moveria el valor
(&self) es inmutable
(&mut self) lo podria cambiar.

self.x //IMPORTANTE!!! SI NO TIENE ';' ES UN RETURN IMPLICITO!!!
}
}

```

En el main.rs:

```

mod fichero;

fn main (){
    let a = fichero::test(); //llama a la funcion externa en fichero y le da el valor 56
    println!("{}", a.get_x());
}

```

También se puede definir una estructura como:

```

struct St1(i32);
struct St2(f32, String, bool);

```

En donde para tomar variables y usarlo sería:

```

let st1 = St1(42);
let st2 = St2(3.2, "hola".into(), true);

```

Y para acceder a sus campos sería con `.0`, `.1`, `.2`, etc...

```

println!("{}", st2.1); //imprimiria "hola"

```

FUNCIONES

Las funciones en Rust estan divididas entre las que no devuelven nada y las que sí.

NO DEVUELVE:

```

fn funcion(num1: &i32){ //referenciada
    println!("El numero es: {}", num1);
}

```

DEVUELVE:

```

fn suma(num1: &i32, num2: &i32) -> i32{ //-> lo que devuelve
    *num1 + *num2 //es importante que no tenga ; ya que este es un return oculto, sin ese punto y coma.
    //Hay que derreferenciar, ya que son referencias (direcciones de memoria)
}

```

```
}
```

Para llamarlas:

```
funcion(&n);  
suma(&x, &y);
```

Para cambiar el valor de un numero:

```
fn cambia(n: &mut i32){  
    *n = 42;  
}
```

y para llamarla:

```
cambia(&mut x);
```

ENUMS

Los enums son tipos de variable como en C++11 y funcionan igual salvo por los métodos que no se pueden hacer en C++:

```
enum Animales{  
    Gato,  
    Perro,  
    Mosca,  
    Ballena,  
}
```

De tal forma que para darle un valor se puede hacer:

```
let animal: Animales = Animales::Mosca;
```

Podríamos crear funciones propias como:

```
impl Animales{  
    fn es_mamifero(&self) -> bool{  
        match self {  
            Animales::Gato | Animales::Perro | Animales::Ballena => true, //"" = 0  
            Animales::Mosca => false  
        }  
    }  
}
```

Siempre habrá que definir cada uno de los Enums en los `match`, por que si no el compilador dará error. A no ser que se utilice el comodín `"_"` que alberga a los restantes.

```
impl Animales{
    fn es_mamifero(&self) -> bool{
        match self {
            Animales::Mosca => false,
            _ => true
        }
    }
}
```

Y para acceder a ese método es como un objeto:

```
let kk = Animales::Gato;
if kk.es_mamifero() {...}
```

ENUMS con variables extra

Los Enums en Rust son como los de C, C++, Java, pero tienen algo único. Se les pueden añadir variables a cada uno de los datos incluidos:

```
enum Status{
    PorHacer,
    EnProgreso{
        asignado_a: String,
    },
    Hecho,
}
```

Y la forma de acceder a ello no es mediante un

```
let estado = Status::EnProgreso;
estado.asignado_a; //error de compilación
```

Sino con un bloque `match`:

```
match Status{
    Status::EnProgreso{ asignado_a } => { println! "Lo hace: {}", asignado_a; },
    _ => {} //si no queremos que haga nada, pues doble llave.}
```


PERO CUIDADO!!!! Si hacemos eso destruimos la variable instanciada de Status, por que el Ownership pasaría en el bloque `{println!...}` al `match` y por lo tanto al querer usar el `Status` de la instancia después daría error de compilación. Para ello es mejor hacer:

`match &Status{ //mismo código}` y ahí se podría usar sin problemas después.

También se puede hacer el acceso con el `if let`.

```
impl Ticket{  
    pub fn assigned_to(&self) -> &str{  
        if let Status::EnProgreso { asignado_a : kk } = &self.status { kk }  
        else { panic!();}  
    }  
}
```

Es complicado de entender pero ahí va:

cuando hay un `let` siempre es una asignación a una variable. pero podemos hacerlo dentro de un `if`. La forma de leer esto es de derecha a izquierda, por que como se puede ver hay una `=` no `==`. entonces esto se estructura así:

`if let PLANTILLA (nueva variable) = algo que existe o no.`

así que se leería, que si esto: `self.status` existe y se adapta a que sea un `Status::EnProgreso { asignado_a }`, entonces mete ese `asignado_a` dentro de la variable `kk` y devuelve ese `kk` de esta forma `{ kk }`. El meter ese valor, se llama DESTRUCTURACION. Sacar el valor de la caja y meterlo dentro.

Por qué es necesario? por que quizá podríamos pensar hacer un:

`if Status::EnProgreso.asignado_a == {asignado_a = paco}` pero primero no sabemos a quien está asignado, y segundo que pasaría si no existe ese `EnProgreso` y es un `Status::Hecho`? no existiría el `.asignado_a`, y por lo tanto daría un `segfault`.

Todo se podría haber resumido así:

`if let Status::EnProgreso { asignado_a } = &self.status { asignado_a } else....`

Pero importante. Fijarse lo que devuelve:

```
pub fn assigned_to(&self) -> &str{
```

Es decir una dirección de memoria (`char*`) por lo tanto el segundo `asignado_a` o el `{kk}` primero, son direcciones de memoria y está bien. Si devolviéramos valores, entonces tendría que ir como `*asignado_a`.

```
enum Shape{
    Circle { radius: f64}, Square { border: f64 }, Rectangle { width: f64 },
}
impl Shape {
    pub fn radius(&self) -> f64{
        if let Shape::Circle { radius } = &self { *radius } //dereferencia por ser &self
        else { panic!(); }
    }
}
```

FUNCIONES CLOSURE |argumentos de funcion |

Hay veces que queremos pasar una funcion mas limpiamente o mas corta.
Se puede definir la normal como:

```
fn sumar (x: i32, y: i32) -> i32{
    x + y
}
```

Pero podemos hacerla Closure (o función anónima):

```
let sumar = |x, y| x + y;
```

y se la llama como:

```
let resultado = sumar(2, 3);
```

esto mismo se puede hacer en una unica linea asi:

```
let sumar = (|x, y| x + y)(2, 3);
```

Donde los argumentos que se le pasan están entre `| |` y los parametros están entre paréntesis.

TRAITS

SOBRECARGA DE OPERADORES

Rust tiene sobrecarga de operadores, y la forma de declararlos es en el `impl` de tal forma:

```
use std::ops::Add; //importa el trait (el operador) de Add. Si no se puede usar. Para implementar muchos poner:
```

```
use std::ops::{Add, AddAssign, Sub, SubAssign, Mul, MulAssign, Div, DivAssign};
```

// Es muy útil derivar Copy y Clone para tipos matemáticos simples hay que hacerlo copy por que sino será un clone que pierde su propiedad y no se puede usar al pasarlo a dicha función.

```
#[derive(Debug, Clone, Copy)]
```

```
struct Vector3{...}
```

```
impl Add for Vector3{
```

```
    type Output = Vector3; //el resultado de la operacion v1 + v2 sera un Vect3
```

```
    fn add(self, other: Vector3) -> Vector3{...}
```

Como se ve, necesita el tipo de output, es decir lo que va a devolver, por que podría ser cualquier otra cosa. Pero en los de asignación no hay que ponerlo:

```
impl AddAssign for Vector3{
```

```
    fn add_assign(&mut self, other: Vector3){
```

```
        self.x += other.x;
```

```
        self.y += other.y;
```

```
        self.z += other.z;
```

```
    }
```

```
}
```

La sobrecarga es con nombre mayúscula, y la función en minúscula pero llamada igual. La lista es la siguiente para las sobrecargas (entre paréntesis el nombre a la función):

+ -> Add (add)

+= -> AddAssign (add_assign)

- -> Sub (sub)

-= -> SubAssign (sub_assign)

* -> Mul (mul)

*= -> MulAssign (mul_assign)

/ -> Div (div)

/= -> DivAssign (div_assign)

% -> Rem (rem)

%= -> RemAssign (rem_assign)

<code>==</code>	<code>-> PartialEq (eq)</code>	<code>!=</code>	<code>-> PartialEq (ne)</code>
<code>[]</code>	<code>-> Index (index)</code>	<code>[] =</code>	<code>-> IndexMut (index_mut)</code>
<code>-x</code>	<code>-> Neg (neg)</code>	<code>!x =</code>	<code>-> Not (not)</code>
<code>&</code>	<code>-> BitAnd (bitand)</code>	<code>&=</code>	<code>-> BitAndAssign (bit_and_assign)</code>
<code> </code>	<code>-> BitOr (bitor)</code>	<code> =</code>	<code>-> BitOrAssign (bit_or_assign)</code>
<code>^</code>	<code>-> BitXor (bitxor)</code>	<code>^=</code>	<code>-> BitXorAssign (bit_xor_assign)</code>
<code><<</code>	<code>-> Shl (shl)</code>	<code><<=</code>	<code>-> ShlAssign (shl_assign)</code>
<code>>></code>	<code>-> Shr (shr)</code>	<code>>>=</code>	<code>-> ShrAssign (shr_assign)</code>
<code>*x</code>	<code>-> Deref (deref)</code>	<code>*x=</code>	<code>-> DerefMut (deref_mut)</code>
<code>x..y</code>	<code>-> Range (no hay)</code>	<code>()</code>	<code>-> Fn, FnMut, FnOnce (call)</code>
<code><, <=, >, >=</code> <code>-> PartialOrd (partial_cmp)</code>			

Para alterar el orden por ejemplo en una multiplicacion de un Vector3 por ejemplo seria:

`2 * v1`

//como se ve el int va a la izda, pero aquí se cruza a la dcha del for por lo que el self, se refiere al tipo int, y por eso no es self.x o self.y.

```
impl Mul<Vect3> for i32{
    type Output = Vect3
    fn mul(self, other: Vect3) -> Vect3{
        return Vect3 {x: self * other.x, y: self * other.y, z: self * other.z}; //hay return, entonces ;
    }
}
```

`v1 * 2`

//aquí el numero está a la derecha, por lo que se cruza y va a la izquierda, y en este caso el for es del Vect3 por lo que el self será self.x, self.y o self.z

```
impl Mul<i32> for Vect3{
    type Output = Vect3;
    fn mul(self, n: i32) -> Vect3{
        Vect3 {x: self.x * n, y: self.y * n, z: self.z * n} //no hay return no hay ; El self tiene que ser en
        minúsculas si se devuelve el valor como aquí.. Podríamos haber puesto:
        ft mul(self, n: i32) -> Self{. Donde ahí si que va en Mayúsculas por que es un tipo.
    }
}
```

El formato es:

```
impl Trait<RightHandElement> for LeftHandElement
```

entonces el self es del tipo del LeftHandElement

SOBRECARGA DE OPERADORES por instancia

En la sobrecarga hemos visto que teníamos que hacer un:

`#[derive(Debug, Clone, Copy)]` al principio para que no pasara los datos de ownership a la función y así tenerlo después disponible. Esto se puede hacer por referencia en vez de copy, sin esta instrucción. En un Vector3 no merece la pena, pero sí en estructuras muy grandes como 1000x1000 por velocidad:

```
use::std::ops::Add;
```

```
struct Vect3{ todo lo de x y z pero sin ser pub f32,}
```

```
impl<'a, 'b> Add<&'b Vect3> for &'a Vect3{ //<' le indica que será un lifetime, es decir que vivirá el tiempo suficiente mientras que esté la función aplicada de Suma. Después de
```

impl lo declaramos y luego después lo usamos. en &'a y &'b. Es por motivos de ownership.

```
    type Output = Vect3;
```

```
    fn add(self, other: &Vect3) → Vect3{
```

```
        x: self.x + other.x, ....
```

```
    }
```

```
}
```

y para usarlo sería :

```
let v3 = &v1 + &v2;
```

```
println!("{}", v1.x); //aquí ya no daría error el compilador por que todavía existe.
```

TRAITS. Para el uso de Templates en sobrecarga operadores

Hemos visto que se puede hacer la sobrecarga de operadores de la multiplicación por un número. Pero y si queremos que pueda ser de cualquier tipo que sean números. Usamos los templates como en C++:

```
//Hay que añadir la dependencia de "num-traits" en el Cargo.toml. Debajo de [Dependencies] poner
```

```
//num-traits = "0.2"
```

```
use std::ops::Mul;
```

```
use num_traits::Num; //esto es un TRAIT e incluye cualquier número (i32, f64, u8, etc..)
```

```
//1. Hacemos que el struct acepte cualquier tipo T que sea un número:
```

```
pub struct Vect3<T>{pub x: T, pub y: T, pub z: T,}
```

//2. implementamos la multiplicación para cualquier número T

```
impl<T> Mul<T> for Vect3<T>
```

where

```
//esto es la condición y nos dice que T tiene que ser un número y poder copiarse
```

```
T: Num + Copy, //fijarse que es , no ; el + indica "and"
```

//si hubieramos querido hacerlo con un solo tipo, podriamos heberlo metido, pero también asi:

//T: Mul<f32, Output = T> + Copy //Cuantas más lineas como esta más restrictivo. Mul<f32> es que se pueda multiplicar por un float. y Output = T indica que el resultado sea uno de Tipo T.

```
{
```

```
type Output = Vect3<T>; //la salida tiene que ser un struct del mismo tipo
```

```
fn mul(self, n: T) -> Vect3<T> {
```

```
    Vect3{
```

```
        x: self.x * n, //Rust ya sabe que se puede multiplicar por que es un Num
```

```
        y: self.y * n, //etc....
```

```
    }
```

```
}
```

```
}
```

TRAITS. Añadir un nuevo método a un tipo

Como hemos visto, podemos hacer sobrecarga de operadores para un nuevo tipo de variable que tengamos. Para ello usábamos unos traits ya definidos que los cargábamos con:

```
use std::ops::{Add, Sub, ...};
```

Pero y si quisieramos construir nuestros propios métodos a dichos tipos nuevos O A LOS YA STANDARD?

```
trait <nuevo_trait> {fn <método> (parametros);}
```

de tal forma que quedaría para saber si un numero i32 es par:

```
trait EsPar{ fn es_par(self) -> bool;}
```

```
impl EsPar for i32{
```

```
    fn es_par(self) -> bool{
```

```
        self % 2 == 0 //devolvera true o false.
```

```
    }
```

```
}
```

Y ahora lo podríamos implementar como:

```
if 42i32.es_par() {...}
```

Hay que tener en cuenta que esto solo lo puede ver si está dentro del mismo módulo (archivo o no dentro de un "mod") si no habría que usar use. Por ejemplo imaginemos que implementamos `es_par()` en otro archivo paralelo al `main.rs` que se llama "`tools`" pues en `main` tendríamos que poner:

```
mod tools;
```

```
use tools::EsPar;
```

y ahí ya sí que sería visto.

si hicieramos un:

```
trait....
```

```
impl EsPar
```

```
mod OtroCodigo{
```

```
    use super::EsPar; //necesario ya que estamos en otro módulo.}
```

TRAITS. Implementar el `from` e `into` a un tipo nuevo

Para que se pueda usar el método `into` y `from` que son duales, por lo tanto al implementar el `From`, queda implementado el `Into`, no tenemos que hacer un

`trait From...` ya que ya está incluido en el "prelude" de Rust, es decir las herramientas básicas de Rust. Por lo tanto no tenemos que usar siquiera un:

```
use std::convert::From
```

Recordemos que para convertir de un `&str` a `String` habia que hacer:

```
let s = "hola".into();
```

o

```
let s = String::from("hola");
```

Ahora para implementarlo para un nuevo tipo:

```
pub struct WrappingU32{
```

```
    value: u32;
```

```
}
```

hemos de hacer esto:

```
impl From<u32> for WrappingU32{
```

```
    fn from(num: u32) -> Self{
```

```
        Self{value: num}
```

```
}
```

```
}
```

y con eso ya podremos hacer cosas como:

```
let kk: WrappingU32 = 42.into();
```

```
let kk = WrappingU32::from(42);
```

TRAITS. Display, debug y Error

Podemos hacer que nuestras funciones devuelvan un `Result(Ok, Err)` de tal manera que al sacar el resultado tengamos que hacer un `unwrap()` o un `match` para dilucidar cual de los dos han sido, pero podemos hacerlo de forma más cómoda implementando nuestros `traits` de `Display`, `Debug` y `Error`.

`Debug` se autogenera cuando se le mete el `#[derive(Debug)]`, que es el que suelta los errores y los tipos para informar en el log del programa por debajo.

pero si queremos informar al usuario de un error en tiempo de ejecución usamos el `Display`, y como este es más personalizable, entonces es necesario implementarlo a mano. Ejemplo:

```
use std::fmt; //para usar el Display
```

```
use std::error::Error; //para usar el Error
```

```
//1. definimos nuestro tipo de error y le añadimos el Debug. El tipo podría ser un Enum
```

```
#[derive(Debug)]
```

```
struct EdadInvalidaError;
```

```
//2. implementamos Display para el usuario final
```

```
impl fmt::Display for EdadInvalidaError {
```

```
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
```

```
        write!(f, "Lo siento, debes ser mayor de 18 años para entrar.") //ver write apuntes.
```

```
    }
```

```
}
```

```
//3. Implementamos el trait Error ya que tenemos Debug y Display (necesarios los dos)
```

```
impl Error for EdadInvalidaError {} //por defecto y con ello ya puedo hacer el operador ?
```

```
//Función que puede fallar
```

```
fn verifica_entrada(edad: i32) -> Result<&'static str, EdadInvalidaError> {
```

```
    if edad >= 18 { Ok("Bienvenido al club!") }
```

```
    else { Err(EdadInvalidaError) }
```

```
}
```

Todo esto para qué? Se verá en el main:

```
fn main() {
```



```
let resultado = verifica_entrada(16)?;
```

//si es error, lo imprimimos. verifica_entrada devolverá un Result. Con el if let, podemos

//...desempaquetarlo. Recordemos se lee dcha a izda. Resultado encaja en Err(error_detectado)? pues mételo en la variable "error_detectado" para luego usarlo.

```
if let Err(error_detectado) = resultado {
```

```
    //uso de Display: Msg bonito para el cliente
```

```
    println!("Error: {}", error_detectado); //imprimirá Error: Lo siento debes ser mayor ...
```

```
    //uso de Debug: imprime informacion para el programador
```

```
    println!("Log: {:?}", error_detectado);
```

```
}
```

```
}
```

Parece que tengo que seguir desempaquetando el tipo de error ya que hago el if let. Pero por qué es necesario en proyectos profesionales de Rust? por que el operador ? no se puede usar sin el trait de error.. y para ello es necesario el de Display.

Entonces como se podría usar el ? sin el if let para desempaquetar?

```
fn main() - > Result<(), EdadInvalidaError> {
```

```
    let resultado = verifica_entrada(16)?; //cuando se usa el ? siempre tiene que devolver un return la función donde se incluye (la main en este caso). Si no, no compila.
```

el ? solo se puede usar si está implementado el Trait Error y Error solo si está Display y Debug.

```
    .... //nunca llegará si la edad es menor de 18
```

```
    Ok(()) //si todo va bien devolverá el Ok.
```

```
}
```

Pero aquí no imprimiría el mensaje de: **Lo siento, debes ser mayor de 18 años para entrar**. Para que lo hiciera tendría que devolver el main otra cosa. Usamos la librería anyhow.

Para ello en [dependencies] del Cargo.toml ponemos

```
[dependencies]
```

```
anyhow = "1.0"
```

y cambiamos el código a:

```
fn main() - > anyhow::Result<()> {
```

```
    let resultado = verifica_entrada(16)?; //se sale e imprime Error: Lo siento, debes...
```

```
    println!("{}", resultado);
```

```
    Ok(())
```

```
}
```

BUCLES

WHILE

El bucle while es:

```
let mut i: u32 = 1;
while i < 10 && i > 0{

    ....

    i += 1;
}
```

La condición nunca va entre paréntesis para evitar la confusión de C ya que en rust la condición es una expresión booleana. Esto es un error:

```
while 5 o while (5)
```

LOOP

loop es infinito a no ser que se saque

```
loop {
    codigo();
    if !condicion {
        break; // asi sale
    }
}
```

FOR

El for en Rust es bajo iteradores

```
for elemento in iterable {...}
```

Por ejemplo:

```
for i in 0..10{ ... } //i va de 0 a 9
for i in 0..=10{ ... } //i va de 0 a 10
```

o con matrices:

```
let v = [1, 2, 3];
for i in v{ ... }
```

Si queremos ir en inversa debemos aplicar el método .rev():

```
for i in (0..10).rev(){ ... } //i va de 9 a 0. Tiene que ponerse los paréntesis para que se forme el grupo primero.
```

COLLECTIONS (contenedores)

Como en C++ tenemos contenedores:

ARRAYS

Podemos usar arrays como en C:

```
let x: [i32; 3] = [1, 2, 3]; //let x = [1, 2, 3]; puede ser así también y el deduciría el tipo.
```

Y si fueran todos el mismo elemento podría definirse como:

```
let x: [i32; 3] = [0; 3]; //x sería [0,0,0]
```

```
println!("{}", x[1]); //imprimiría 2, pero es arriesgado
```

```
println!("{}", x[7]); //Entraría en modo "panic" que saldría de forma segura. No soltaría basura
```

Para hacerlo de forma segura se puede usar

```
x.get(1); //devolvería "Some(2)" en mayúsculas la S
```

```
x.get(7); //devolvería "None" en mayúsculas la N
```

Por lo tanto para usarlo por que un x.get(1) devolvería en pantalla un some(2) con un println! tenemos que hacer:

```
if let Some(i) = x.get(1) {  
    println!("{}", i);  
} else {  
    println!("no existe");  
}
```

O en formato Rust:

```
match x.get(2) {  
    Some(i) => println!("{}", i),  
    None    => println!("No existe"),  
}
```

El `.get` lo que hace es devolver un `OPTION` que puede ser un `Some(T)` es decir algo, o `None`.. es decir nada. Explicado más abajo en la parte de memorias.

1. VECTOR DINAMICO:

```
let mut v: Vec<u32> = Vec::new();  
v.push(1); //mete el 1  
v.push(2); //mete el 2 detrás del 1.  
v.pop(); // quita el 2 (el ultimo) Devuelve un Option<T>
```

Para acceder a los datos se puede usar:

```
v[i]; //puede ser panic  
v.get(i); //seguro.
```

Tambien se puede inicializar el vector en el momento de ser declarado con una macro:

```
let numeros = vec![1, 2, 3]
```

Y se podria inicializar la memoria ya de principio con:

```
let mut numeros = Vec::with_capacity(3);
```

El tamaño es con:

```
v.len();
```

Se puede usar iteradores:

```
let mut it = v.iter(); //el iterador NO SE SITUA en el primer elemento sino ANTES  
it.next(); //es el siguiente. En este caso seria ya el PRIMER elemento. PERO LO SUELTA!  
it.next_back(); //es el anterior, PERO EMPEZANDO POR ATRAS y se situa DETRAS del ultimo
```

Los dos devuelven `Option<&T>` por lo tanto para usarlos:

```
if let Some(x) = it.next(){ println!("{}", x)}; //y si fuese el primer .next imprimiria "1"
```

NOTA!! NO SE PUEDE DEFERENCIAR UN ITERADOR. *it no tiene sentido!

Así, para cambiar un valor seria:

```
let mut v = vec![1, 2, 3];  
let mut it = v.iter_mut();  
if let Some(i) = it.next() { *i = 0; } //ahora v es [0, 2, 3]
```

3. STRING

4. HASHMAP

5. VECDEQUE

6. HASHSET

7. BTREEMAP

SLICE

El Slice es como un `String_view` de C++. Es decir una ventana donde podemos acotar el contenedor que tenemos, aunque `string_view` está solo destinado a Strings en C++.

Ello lo que hace, es no hacer una copia, sino guardar en memoria heap, tanto el puntero donde empieza ese recorte y la longitud del mismo.

En Rust, los slices se definen como `&[T]`

//sin recorte declaraciones independientes (no se puede usar una detras de otra)

```
let v = vec![1,2,3,4,5];
```

```
let s1: &[i32] = &v[..]; //toma todo el v sin hacer copia de el.
```

```
let s1: &[i32] = v.as_slice(); //toma todo v sin hacer copia
```

//con recorte

```
let s1: &[i32] = &v[2..]; //s1 = [3,4,5];
```

```
let s1: &[i32] = &v[..3]; //s1 = [1,2,3]
```

ITERADORES

Ya hemos visto en los vectores dinamicos que se pueden usar iteradores:

```
let mut v: Vec<i32> = Vec::new();
```

```
v.push(1);
```

```
v.push(3);
```

```
let mut it = v.iter(); //TYPE: std::slice::Iter<'_, T> donde T es el tipo de lo que se mete
```

Esto situa el iterador al principio del vector (en el 1).

Ese iterador es inmutable (&T) pero se pueden hacer mutables:

```
v.iter_mut() //TYPE: std::slice::IterMut<'_, T>
```

o consume el vector y toma los elementos por propiedad (ownership)

```
v.into_iter(); //TYPE: std::vec::IntoIter<T>
```

Al respecto del into_iter(), podemos implementar un iterador para una estructura como en C++. Por ejemplo en C++ tenemos:

```
struct Ticket { std::string title; };

struct TicketStore {

    std::vector<Ticket> tickets; //instancia

    auto begin() { return tickets.begin(); } //auto = std::vector<Ticket>::iterator

    auto end() { return tickets.end(); } };

int main() {

    TicketStore tienda;

    tienda.tickets.push_back({"hola"});

    tienda.tickets.push_back({"mundo"});

}
```

//ahora gracias a begin() y end() podemos meterlo en un for:

```
//for (it = tienda.begin(); it != tienda.end(); it++){

for (const auto& ticket: tienda) { //por cada ticket que este en tienda.

    std::cout << ticket.title << std::endl;

}
```

Pues en Rust seria:

```
#[derive(Clone)]

pub struct TicketStore{ tickets: Vec<Ticket> }

#[derive(Clone, Debug, PartialEq)]
```



```
//o directamente la linea self.tickets.into_iter() sin ;
```

```
}
```

A partir de ello se podria meter en un bucle for:

```
fn main(){  
    let mut tienda = TicketStore::new();  
  
    let mi_ticket = Ticket {  
        title: "hola".into(),  
        description: "".into(),  
        status: Status::ToDo,  
    }  
  
    tienda.add_ticket(mi_ticket);  
  
    for ticket in tienda {  
        println!("Ticket: {:?}", ticket.title);  
    }  
}
```

si nos fijamos en la Firma de la funcion `into_iter()`

```
(fn funcion(iter: std::vec::IntoIter<Ticket>)  
  
impl IntoIterator for TicketStore {  
    type Item = Ticket;  
  
    type IntoIter = std::vec::IntoIter<Ticket>;  
  
    fn into_iter(self) -> Self::IntoIter { //devolvemos un iterador  
        self.tickets.into_iter()  
    }  
}
```

para definir `iter()` seria `(fn funcion(iter: std::slice::Iter<'_, Ticket>)):`


```
// implementamos el trait para la REFERENCIA (&TicketStore)
```

```
impl<'a> IntoIterator for &'a TicketStore {
```

```
    type Item = &'a Ticket; // Ahora devuelve referencias, no el Ticket entero
```

```
    type IntoIter = std::slice::Iter<'a, Ticket>; // La máquina de Vec para referencias
```

```
    fn into_iter(self) -> Self::IntoIter {
```

```
        self.tickets.iter() // Delegamos en .iter() del vector
```

```
    }
```

```
}
```

para definir `iter_mut()` seria (`fn funcion_mut(iter: std::slice::IterMut<'_, Ticket>)`):

```
// Implementamos para la REFERENCIA MUTABLE (&mut TicketStore)
```

```
impl<'a> IntoIterator for &'a mut TicketStore {
```

```
    type Item = &'a mut Ticket; // Devuelve referencias mutables
```

```
    type IntoIter = std::slice::IterMut<'a, Ticket>; // La máquina para mutables
```

```
    fn into_iter(self) -> Self::IntoIter {
```

```
        self.tickets.iter_mut() // Delegamos en .iter_mut() del vector
```

```
    }
```

```
}
```

ITERADORES. Combinators y Collect

<https://doc.rust-lang.org/std/iter/trait.Iterator.html#method.filter>

Los iteradores tienen métodos, que se llaman combinators. El mas famoso ya introducido es `.next()` que avanza hasta el primero y luego el segundo, etc...(como un `it++`) o un `.next_back()` que avanza desde el final hacia el último. Esos son los más importantes, pero hay otros como:

```
let numeros = vec![1, 2, 3, 4, 5];
```

```
//sumamos SOLO los números PARES
```

```
let solucion: u32 = numeros.iter()
```

```
.filter(|&n| n % 2 == 0) //selecciona solo los pares. es una función Closure

.map(|&n| n * n) //funciona como una tubería: de los pares los multiplica por si mismos

.sum(); //va sumando todos y al final se mete en solución.
```

Todo esto se podría haber puesto en una línea como:

```
let solucion: u32 = numeros.iter().filter(|&n| n % 2 == 0).map(.....
```

Es decir funciona como una tubería (pipe). Pero el valor se almacena en una variable pero se puede perder. La forma de almacenarlo en un Vector que va haciendo reserva de memoria en el heap es con `.collect()`

```
let solucion: Vec<u32> = numeros.iter()

    .filter(|&n| n % 2 == 0) //selecciona solo los pares. es una función Closure

    .map(|&n| n * n) //funciona como una tubería: de los pares los multiplica por si mismos

    .collect(); //Los mete en un vector
```

O

```
let solucion = numeros.iter()

    .filter(|&n| n % 2 == 0) //selecciona solo los pares. es una función Closure

    .map(|&n| n * n) //funciona como una tubería: de los pares los multiplica por si mismos

    .collect::<Vec<u32>>; //TurboFish ya que se parece a pez -> ::<>
```

ITERADORES como argumentos función. impl Trait.

Hemos visto que podríamos recibir en una función un iterador para hacer con el lo que quisiéramos:

```
para .iter(): fn funcion(iter: std::slice::Iter<'_, T>);

para .iter_mut(): fn funcion(iter: std::slice::IterMut<'_, T>);

para .into_iter(): fn funcion(iter: std::vec::IntoIter<T>);
```

Pero claro, dicha función solo admitiría un iterador de un VECTOR, por que así son sus firmas. las de `VecDeque` son `std::collections::vec_deque::Iter` y las de `LinkedList` son: `std::collections::linked_list::Iter`

Si quisieramos aceptar el de cualquiera:

para `.iter()`: `fn funcion(iter: impl Iterator<Item = &T>);` `//.iter()` da un `&`

para `.iter_mut()`: `fn funcion(iter: impl Iterator<Item = &mut T>);` `//da un &`

para `.into_iter()`: `fn funcion(iter: impl Iterator<Item = T>);` `//da por valor`

RESERVA DE MEMORIA EN EL HEAP

No se puede. Rust lo prohíbe. De hecho esa es la protección de Rust frente a las fugas de memoria, punteros colgantes etc. Cuando nosotros hacemos una estructura como:

```
pub struct Estructura{
    dato1: u32,
    dato2: String,
}
```

```
fn main (){
```

```
    //Rust no deja hacer esto:
```

```
    //let mi_estructura: Estructura; por que contendría valores basura. Algo prohibido en Rust.
```

```
    //Sino que hay que inicializarla de esta manera:
```

```
    let mi_estructura = Estructura{
```

```
        dato1: 32, //se asigna 4 bytes en el STACK ya que sabe cuanto son y es más rapido.
```

```
        dato2: "hola".into(), //se reserva 4 bytes en el HEAP y 24 bytes en el STACK con los metadatos
                                (que incluyen la direccion de memoria, el numero de bytes ocupados y el num de bytes de capacidad.
```

```
    }
```

```
} //al salir del main se hace el free automaticamente de la memoria reservada en el HEAP y el STACK
```

Para elementos que no sabemos cuanto van a ser de grandes, como `String` o `Vector`, se hace esto:

```
let mut s = String::with_capacity(5);
```

`//aquí reservamos 5 bytes en el HEAP para que pueda crecer y en el STACK están los metadatos del String con 8 bytes para la dirección de memoria inicial en el HEAP, la`

longitud actual en número entero que sería cero por ahora (8 bytes más) y la capacidad que son otros 8 bytes.

Para los vectores, por ejemplo capacidad para 5 numeros enteros:

```
let mut v = Vec<i32> = Vec::with_capacity(5); //STACK (8bytes memoria, 8bytes 0longitud, 8bytes 5 capacidad.
```

```
v.push(12); //la longitud pasa a ser 1 y la capacidad sigue siendo 5.
```

En ambos como en el caso de la estructura, al salir de su scope {}, se hace el free automatico.. pero se podría forzar un free con esta instrucción antes de terminar las llaves:

```
drop(v);
```

```
drop(s);
```

TIEMPO DE VIDA. LIFETIME. OWNERSHIP!!!

Rust para preservar la memoria, tiene un método que es el **ownership**. Si por ejemplo hacemos esto:

```
let num = 32;  
ft_funcion(num);  
println!("{}", num);
```

No hay problema por que los tipos básicos, tienen implementando **Trait** el **Copy** o **Clone** de los mismos y por lo tanto el compilador no dará problema.

Pero si tenemos un:

```
struct MiStruct{ dato: i32 };  
let kk = MiStruct {dato: 42 };  
  
ft_funcion(kk); //hemos perdido el ownership aqui. Ya no es dueña kk de los datos. Se han transferido a la función  
  
println!("{}", kk.dato); //da error aqui por que no tiene el ownership ya.
```

Esto se puede arreglar con una sola linea y es implementando el **Trait** de **Copy Clone** para la estructura.. y se tiene que hacer arriba de ella:

```
#[derive(Copy, Clone)]  
struct MiStruct{ dato: i32, };  
  
.... //el resto del código.
```

Ahora no daría error de compilación ya que la estructura al llamarse a la función, se copiaría.

Si la estructura tuviera un String dentro, el Copy no se puede hacer ya que parte de los datos del String se almacenan en el Heap, y por lo tanto hay que aplicar un **.clone()**:

```
#[derive(Clone)]
```

```
struct MiStruct{ dato: String };
let kk = MiStruct {dato: "hola mundo".into(),};
ft_funcion(kk.clone()); //y ahí pasamos un clone y no se pierde.
```

Esto mismo pasaría si en vez de a una función se le pasa a un match, o cualquier cosa que esté entre llaves {}.

Para no hacer copia se puede pasar la referencia como en C++:

```
struct MiStruct{ dato: i32 };
let kk = MiStruct {dato: 42 };
ft_funcion(&kk); //donde la funcion recibe un & tambien como fn ft_funcion(kk: &MiStruct);
println!("{}", kk.dato); //ya no da error.
```

Y lo mismo pasándolo a un match:

```
enum Status{ PorHacer, EnProgreso{ asignado_a: String,}, Hecho,}
match &Status{ //si no pusiera el & en el => { println!..., asignado_a perdería el ownership
    Status::EnProgreso{ asignado_a } => { println! "Lo hace: {}", asignado_a; },
    _ => {} //si no queremos que haga nada, pues doble llave.}
```

Pero ¿qué está haciendo el compilador por debajo? está dándole una vida a la referencia en el bloque de la llamada a la función de tal forma que cuando hago un `ft_funcion(&kk)`; la estructura `kk` va asegurarse a vivir durante TODO el tiempo de la función, aunque luego se haga llamada a otras funciones:

```
struct ST {dato: String}; //debe estar fuera para ser reconocida por funcion1 y funcion2
fn main(){
    let st = ST { dato: "hola".into() };
    funcion1(&st); //no pierde el ownership. Sigue perteneciendo st a main.
    println!("{}", st.dato); //compila sin problema por que no perdió el ownership.
}
```

//En Rust no hace falta declarar las funciones antes de usarlas.

```
fn funcion1(st: &ST){
    //al loro con esto!! ahora st es una direccion de memoria asi que no se pasa como funcion2(&st) que
    daría un error de compilación.
    funcion2(st);
    println!("{}", st.dato); //compila
}
fn funcion2(st: &ST){
```

... //dentro de la primera llamada en el main al hacer & Rust hace que st viva hasta aquí y después de cada línea de la función.

```
}
```

LIFETIMES!!

Cuando es obligatorio usar LIFETIMES? (<'loquesea>) con una referencia &?

1. Cuando guardamos una referencia en un STRUCT o ENUM:

```
struct ST { texto: &str } //&str es como un char*. pero ERROR!! Rust no sabe cuanto vive
```

Para Solucionarlo:

```
//primero se define el lifetime y luego se aplica. el ST<'a> va a tener un str que vive 'a y no menos.
```

```
struct ST<'a> {texto: &'a str, } //Significa que ST no puede vivir más que el texto str, por lo que si no tendríamos un puntero basura.
```

2. Cuando pasamos varias referencias que devuelve una de ellas. Si una función recibe dos o más referencias y devuelve otra referencia, el compilador se confunde y no sabe cual de los parámetros proviene del dato resuelto.

```
//Rust no sabe si el &str viene de x o y, por lo que una podría vivir menos que otra y contendría basura.
```

```
fn el_mas_largo(x: &str, y: &str) -> &str //error de compilación
```

Para solucionarlo:

```
fn el_mas_largo<'a>(x: &'a str, y: &'a str) -> &'a str {  
    if x.len() > y.len() { x } else { y } //la referencia devuelta vivirá tanto como la menor.  
}
```

OPTION

Qué pasa en C cuando tenemos un `Char* str = NULL`, y le hacemos un `str[0]`, pues que se produce un Segfault y para evitarlo hacemos un `if (str) {y ahora sí podemos acceder a str[0]...}`

Es decir, si no chequeamos que primero existe `str`, entonces se puede producir el crash.

Rust evita esto, devolviendo en muchos casos y métodos un tipo llamado `OPTION`, que en sí es un enum:

```
enum Option<T>{  
    Some(T),  
    None,  
}
```

Es decir devuelve una Tupla de valores. Un **Some(T)** si devuelve "algo" (y **Some** es palabra reservada para ello) o un **None**, es decir nada.

Esto puede ser un número o algo.. o una dirección de memoria o un **NULL**. De esa manera siempre estamos protegidos.

Para usarlo puede ser así:

Recordando la estructura **Ticket** que tenía un **status** de **enum Status** con **assigned_to** en el de **InProgress**:

```
pub fn assigned_to(&self) -> Option<&String>{ //ya no tendremos que hacer panic ya que la otra opción sería un None (null).
```

```
if let Status::InProgress { assigned_to } = &self.status { Some(assigned_to) }  
else { None }
```

RESULT

Es la otra tupla de Rust, y en este caso es:

```
enum RESULT<T, J>{  
    Ok(T),  
    Err(J),  
}
```

En este caso estamos viendo el resultado de una operación o paso a función que queramos probar.

```
fn dividir (dividendo: i32, divisor: i32) -> Result <i32, String>{  
    if divisor == 0 {  
        Err("No se puede dividir por cero!".to_string())  
    }  
    else{  
        Ok(dividendo / divisor)  
    }  
}
```

```
fn main{  
    let resultado = dividir(10, 2); //no podemos mostrar resultado por que es un "Result" hay que desempaquetarlo con un match  
    match resultado{  
        Ok(i) => println!("El resultado es: {}", i),  
        Err(str) => println!("{}", str),  
    }  
}
```

También en vez del `match` que es muy largo, se podría haber puesto:

```
let resultado = dividir(10,2).unwrap(); //si es Err entonces hay un panic.  
println!("El resultado es: {}", resultado); //con unwrap() hemos sacado el valor del Ok, si llega a ser Ok.
```

Y ese panic hubiera salido CON el msg de error definido en la función. Si queremos uno más personalizado podríamos hacer:

```
let resultado = dividir(10,2).expect("No se puede dividir por cero"); //saldrían los dos errores de texto y también hace panic.
```

MACROS

Las Macros en Rust son como magia negra. Componen código a partir de lo que se les pasa y realizan acciones o devuelven valores dependiendo de ello. Todas ellas son activadas cuando en la palabra clave, se le pone un `!` al final. Las más famosas:

`print!()`

Imprime el texto en pantalla sin salto de línea. El texto puede quedarse esperando en el buffer de la consola y no aparecer en pantalla hasta que se imprima un salto de línea:

```
fn main(){  
    let str: &str = "hola ";  
    print!("{}", str); //el {} es como el %s del printf, pero no hace falta poner el tipo.  
    print!(" mundo!");  
    //En pantalla se verá "hola mundo!" pero por que termina el scope de la función. Si no seguiría sin imprimir.  
}
```

Para soltarlo en pantalla si queremos antes de finalización de Scope se necesita el `flush()`

```
use std::io::{self, Write}; //necesario para el flush()  
fn main(){  
    print!("hola");  
    print!(" mundo!");  
    io::stdout().flush().unwrap(); //sin esto no imprimiría. Unwrap() por que devuelve un Result  
    loop{} //infinito, por lo que impediría que se escribiera en pantalla a no ser del flush().  
    Mejor que un while true{} por optimización de código.  
}
```

Sin el `flush()` si pusieramos `print!(" mundo!\n");` sí que lo imprimiría, pero para eso usamos uno `println!()`

println!()

La diferencia es que imprime el salto de linea y por lo tanto vacia el buffer:

```
fn main(){
    let str: &str = "mundo!";
    println!("hola {}, str); //En pantalla se verá "hola mundo!" aunque haya un loop infinito después
    loop{}
}
```

format!()

No imprime nada. Es como el Ostringstream de C++ para no ir reajustando la memoria en cada + que le hagamos al String. Devuelve un String siempre:

```
fn main(){
    let nombre = "Paco"; //aquí nombre es un &str (char*) estático
    let msg = format!("hola, {}!", nombre); //aquí msg es un String.
    println!("{}", msg); //para imprimirlo.. Si no no hará nada y al salir del scope liberará memoria.
}
```

write!()

No imprime nada en pantalla por si solo. Escribe en variables, archivos o red.

1. Como ostringstream de c++. Escribir a un String:

```
use std::fmt::Write; //Obligatorio para escribir en Strings
fn main(){
    let mut buffer = String::from(""); //aunque sea vacio ya existe.
    write!(&mut buffer, "paso {}, paso{}.", 1, 2).unwrap(); //Devuelve un Result así que
    unwrap() para desempaquetarlo
    println!("{}", buffer); //imprimirá "paso 1, paso 2."
}
```

2. Escribir en un Archivo:

```
use std::fs::File;
use std::io::Write as IoWrite; //obligatorio para escribir en archivos/sockets
fn funcion(){
    //creamos el archivo en el disco duro
```

```
let mut archivo = File::create("log.txt").unwrap(); //unwrap ya que es un result.  
//mete directamente el texto en el archivo. No en pantalla  
write!(&mut archivo, "Error crítico en el año {}, 2026).unwrap();  
}
```

panic!()

Ya tratado más arriba en los apuntes.

todo!()

Sirve para indicarnos que debemos hacer algo en una función o lo que sea. Saltará un Panic() cuando llegue.

unreachable!()

Cuando estemos 100% seguro de que el flujo del programa nunca debería llegar a ese punto. Si por algún motivo llegase, lanzará un panic:

```
let semaforo = "azul";  
match semaforo {  
    "verde" => avanzar();  
    "rojo" => detener();  
    _ => unreachable!(); //como es azul hará panic()  
}
```

unimplemented!()

Cuando no tenemos intención de programar esa parte de código:

```
fn pagar_con_cheque(){  
    unimplemented!(); //el programa no soporta cheques. Si se llama panic!  
}
```

assert_eq!(valor a, valor b)

Sirve para comprobar que dos valores son exactamente iguales. Si lo son, el programa continua en silencio. Si no, se produce un **panic!**

```
fn sumar(a: i32, b: i32) -> i32{ a + b }  
fn main() {  
    let resultado = sumar(2, 2);
```

```
assert_eq!(resultado, 4); //pasará en silencio
```

```
assert_eq!(resultado, 5, "Error: el resultado real debería ser {}, resultado); //panic!  
se puede poner con o sin mensaje de error que saltará en el panic!
```

```
}
```

Cuando tenemos un tipo propio como un **Struct** debemos añadirle al **Struct**, **Enum** o lo que sea un **#[derive(PartialEq, Debug)]**, para que pueda hacer sus comprobaciones.